Phase 11 — Kubernetes (Complete Notes)

> *Everything you did by hand in Phases 5–10 — restarts, scaling, load balancing, rollouts — declared in YAML and automated by a control loop.*

---

1. Why Kubernetes exists

Docker Compose runs your stack on ONE machine. At many machines, questions explode: which server runs which container? Who restarts a crashed one at 3am? How do containers find each other across hosts? How do you roll out v2 across 40 containers with zero downtime? Scale up at peak?

Kubernetes (K8s) is a **container orchestrator**: you declare the desired state ("3 replicas of api:1.2, each needs 512Mi, expose on 8080"), and its control loops continuously **reconcile reality toward that declaration**. Server dies → pods rescheduled elsewhere. Pod crashes → restarted. That's the paradigm shift: **imperative (do X) → declarative (be X)**.

Analogy: compose = you personally manage one restaurant's kitchen. K8s = a franchise operations HQ: you publish the standard ("every branch: 3 cooks, this menu"), HQ continuously staffs, replaces sick cooks, opens capacity at rush hour — across all branches, without calling you.

Honest counterweight: K8s is complex. One VPS + compose (Phase 7) is *correct* for small systems. K8s earns its cost with: many services, many teams, elastic load, or when you inherit it at work (the practical reason you're learning it).

2. Architecture

```
CONTROL PLANE (the brain — managed for you in EKS/GKE/AKS)
├── API server   — the ONLY door; kubectl talks to this
├── etcd         — key-value store holding ALL cluster state
├── scheduler    — decides WHICH node each new pod lands on (resources, constraints)
└── controllers  — the reconciliation loops (desired vs actual, forever)

WORKER NODES (the muscle — your EC2 instances / VMs)
├── kubelet      — node agent: starts/stops pods as told, reports health
├── kube-proxy   — makes Services work (routing/iptables)
└── container runtime (containerd) — actually runs containers
```

3. Core objects (the vocabulary — learn these cold)

Pod

Smallest deployable unit: **one or more containers** sharing network (one IP, localhost between them) and volumes. Usually 1 app container (+ maybe a sidecar). **Pods are mortal cattle**: they die, get rescheduled, get new IPs. You almost never create pods directly →

Deployment

Declares: image, replica count, update strategy. Manages ReplicaSets which manage pods. Gives you: self-healing replicas, **rolling updates** (`kubectl set image` → new pods up, old drained down, automatically — your Phase-10 manual dance, built in), and **rollback** (`kubectl rollout undo`).

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop-api }
spec:
  replicas: 3
  selector: { matchLabels: { app: shop-api } }
  template:                                # pod template
    metadata: { labels: { app: shop-api } }
    spec:
      containers:
      - name: api
        image: registry.io/shop-api:1.4.2        # never :latest (Phase 6!)
        ports: [{ containerPort: 8080 }]
        envFrom:
        - configMapRef: { name: shop-config }
        - secretRef:    { name: shop-secrets }
        resources:
          requests: { cpu: 250m, memory: 512Mi }   # scheduler placement + HPA math
          limits:   { cpu: "1",  memory: 1Gi }     # kill/throttle ceiling
        readinessProbe:                            # "may I receive traffic?"
          httpGet: { path: /actuator/health/readiness, port: 8080 }
          periodSeconds: 5
        livenessProbe:                             # "am I alive, or restart me?"
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          initialDelaySeconds: 30
```

★ Readiness vs liveness is a classic interview + production topic: readiness gates *traffic* (fail → removed from Service endpoints — like LB health checks, Phase 10); liveness gates *existence* (fail → container restarted). Wrong liveness (checking the DB!) = restart storms during a DB blip.

Service — stable networking over mortal pods

Pods' IPs churn; a **Service** gives a fixed virtual IP + **DNS name** and load-balances over pods matching its label selector (Phase-2 DNS + Phase-10 LB, inside the cluster):

```
apiVersion: v1
kind: Service
metadata: { name: shop-api }
spec:
  selector: { app: shop-api }          # matches pod labels
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP                      # internal-only (default)
```

Any pod can now call `http://shop-api` (or `shop-api.default.svc.cluster.local`). Types: **ClusterIP** (internal), **NodePort** (dev exposure on every node's port), **LoadBalancer** (cloud provisions a real NLB/ALB per service — pricey per-service, hence →Ingress).

ConfigMap & Secret — config out of images (12-factor, Phase 6 lesson)

```
kubectl create configmap shop-config --from-literal=SPRING_PROFILES_ACTIVE=prod
kubectl create secret generic shop-secrets --from-literal=DB_PASSWORD='s3cret'
```

Same image dev→prod; only mounted config differs. (Secrets are base64, NOT encrypted by default — real setups add encryption-at-rest/External Secrets/Vault.)

Ingress + NGINX Ingress Controller — the cluster's front door (L7)

One entry point routing by host/path to Services — NGINX from Phase 5, reborn as cluster infrastructure:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations: { cert-manager.io/cluster-issuer: letsencrypt }   # auto-TLS!
spec:
  ingressClassName: nginx
  rules:
  - host: api.shop.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: shop-api, port: { number: 80 } } }
  tls: [{ hosts: [api.shop.com], secretName: shop-tls }]
```

Flow: `Internet → cloud NLB → ingress-nginx pods (L7 routing, TLS via cert-manager/Let's Encrypt) → Service → Pods.` The full Phase 5+7 stack — declared in 20 lines, certificates auto-renewing.

4. Auto scaling

- **HPA** (Horizontal Pod Autoscaler): `kubectl autoscale deployment shop-api --min=3 --max=15 --cpu-percent=70` — replicas follow load (metrics-server required; custom metrics like RPS possible). Works because your app is **stateless** (Phase 9 — the payoff).
- **Cluster Autoscaler / Karpenter**: adds/removes actual NODES when pods can't schedule. HPA scales pods; CA scales machines; together: traffic spike → more pods → more nodes → ...and back down (pay less at night).

5. kubectl — the daily toolkit

```
kubectl get pods -o wide / get deploy / get svc / get ingress
kubectl describe pod shop-api-7d9f...      # ★ events at the bottom = why it's broken
kubectl logs -f deploy/shop-api            # logs (add --previous after a crash!)
kubectl exec -it <pod> -- sh
kubectl apply -f k8s/                      # declare everything in a folder
kubectl rollout status deploy/shop-api
kubectl rollout undo deploy/shop-api       # instant rollback
kubectl scale deploy shop-api --replicas=5
kubectl top pods                           # resource usage
kubectl port-forward svc/shop-api 8080:80  # debug: tunnel a service to your laptop
```

Debug decision tree (memorize)

```
Pending    → describe: no resources (requests too big? nodes full?) / no matching node
ImagePullBackOff → image name/tag typo, private registry auth
CrashLoopBackOff → app starts & dies: logs --previous (config? DB unreachable? OOMKilled in describe?)
Running but no traffic → readiness failing? Service selector ≠ pod labels? (classic!) port mismatch?
```

6. Production notes

- Managed control plane (EKS/GKE/AKS) — never self-host etcd as a beginner.
- Everything as YAML in git (GitOps: ArgoCD/Flux syncs git → cluster; `kubectl` becomes read-only for humans).
- Stateful services (PostgreSQL): StatefulSets exist, but default advice remains **RDS/managed DB outside the cluster**; keep the cluster stateless.
- Namespaces per env/team; resource requests on EVERYTHING (scheduling + fairness); PodDisruptionBudgets for safe node drains.

7. Common mistakes

- No resource requests/limits → scheduler flies blind; one leaky pod starves the node.
- Liveness probe hitting the database → DB hiccup restarts the whole fleet (thundering restart storm).
- `:latest` + `imagePullPolicy` confusion → "I deployed but nothing changed".
- Service selector/label mismatch → 0 endpoints → mysterious connection refused (check `kubectl get endpoints`).
- Treating K8s as required: deploying a 2-container hobby app on a 3-node cluster ($200/mo to avoid a $6 VPS).
- Secrets in git as plain YAML.
- Skipping `kubectl describe` and guessing (the events section literally tells you the problem).

8. Interview questions

1. What problem does K8s solve over Docker Compose? What does "declarative + reconciliation" mean?
2. Pod vs Deployment vs ReplicaSet vs Service — roles and relationships.
3. Readiness vs liveness probe; consequences of each failing; why shouldn't liveness check the DB?
4. How does a rolling update achieve zero downtime? How do you roll back?
5. Service types: ClusterIP vs NodePort vs LoadBalancer vs Ingress — when each?
6. How does service discovery work in-cluster (DNS)? What happens when a pod IP changes?
7. HPA vs Cluster Autoscaler? What app property makes HPA safe? (statelessness — Phase 9)
8. Walk through debugging CrashLoopBackOff, then "pods Running but service unreachable".

9. LAB — local cluster

```
# minikube (or kind / k3d)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start --driver=docker
kubectl get nodes

# 1. deploy your Phase-6 Spring Boot image
minikube image load shop-api:1.0                    # local image into the cluster
kubectl create deployment shop-api --image=shop-api:1.0 --replicas=3   # then: switch to YAML (§3)
kubectl expose deployment shop-api --port=80 --target-port=8080
kubectl get pods -w                                 # watch them come up

# 2. self-healing — the magic moment
kubectl delete pod <one-of-them>                    # K8s instantly creates a replacement
kubectl get pods -w                                 # watch the resurrection

# 3. rolling update + rollback
kubectl set image deploy/shop-api shop-api=shop-api:2.0 && kubectl rollout status deploy/shop-api
kubectl rollout undo deploy/shop-api

# 4. ingress + scaling
minikube addons enable ingress metrics-server
# apply an Ingress (host: shop.local + /etc/hosts entry → minikube ip)
kubectl autoscale deploy shop-api --min=2 --max=6 --cpu-percent=50
ab -n 20000 -c 100 http://shop.local/api/products   # then: kubectl get hpa -w  → watch replicas grow

# 5. break things on purpose: wrong image tag, selector mismatch, liveness on a dead path
#    → practice the §5 decision tree on each
```

10. ASSIGNMENT 11 (submit to Claude)

1. Full YAML set for your shop-api: Deployment (3 replicas, resources, both probes, envFrom), Service, ConfigMap, Secret, Ingress with TLS. Working `kubectl get pods/svc/ingress` output.
2. Narrate lab step 2 and 3: what did the controllers actually do, object by object, during self-heal and rolling update?
3. Break-fix report from lab 5: for each of the three sabotages — the symptom, the command that revealed it, the fix.
4. Your pods OOMKill every few hours under load (JVM!). Explain the interplay of `resources.limits.memory` and JVM heap, and the proper fix (hint: -XX:MaxRAMPercentage / container-aware JVM).
5. Argue both sides in ~10 sentences: "We're a 4-dev startup with one Spring Boot monolith at 200 req/s — should we adopt Kubernetes?" Then give your verdict as the senior engineer.